National University of Computer and Emerging Sciences

Lab Manual 10

Programming Fundamentals

Course Instructor	Ms. Anosha Khan
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Fall 2025

FAST - School of Computing
FAST-NU, Lahore Campus

Activities:

Note: Include input validation in programs wherever applicable. Don't use any built-in functions.

1. Implement a function **isEven(int num)** that returns true if the number **num** is even. Then implement the function **isEvenPerfect(int num)** that returns true if the number is an even perfect number. The function **isEvenPerfect(int num)** must call the **isEven(int num)** function. Finally, implement the function **takeEvenPerfectInput(int data[], int size)**. This function takes input into the array from the user and only stores the value in the array if it is an even perfect number. This function must call the **isEvenPerfect(int num)** function. Keep taking input from the user till 'size' number of even perfect numbers are entered by the user. In case of a number that is not even or perfect, ignore it.

   Sample example of `takeEvenPerfectInput(int data[], int size)`:

   size=5

   User input

   1,5,4,9,16,25,16,36,64 stop. 5 even perfect numbers obtained.

   Array data after user input: 4,16,16,36,64

2. Create a program that takes an NxN (where N <= 5), 2D integer matrix, and performs the following transformation:
   - **PowerUpRows**: Multiply each row by its row number. Implement the function:
     void powerupRows(int matrix[][5], int r);

   Example Input (shown for 3x3):

   Matrix:

   4 5 6
   7 8 9
   1 2 3

   Example Output after the function:

   Output Matrix:

   4 5 6
   14 16 18
   3 6 9

3. Write a program that takes a square matrix (2D array) of size N x N (where N <=5) and calculates the sum of its main diagonal and anti-diagonal elements. The main diagonal consists of elements from the top left to the bottom right, while the anti-diagonal consists of elements from the top right to the bottom left. You must define two functions:

   - void inputMatrix(int matrix[][5], int N): takes input in a matrix, after being called from the main() function.
   - void sumDiagonals(int matrix[][5], int N, int& mainDiagSum, int& antiDiagSum): calculates the sum of both diagonals, after being called from the main() function.

   Example Input:

   Enter elements of the matrix:
   1 2 3
   4 5 6
   3 8 4

   After the sumDiagonals Function:

   mainDiagSum = 10, antiDiagSum: 11

4. Create a program that defines three functions:
   - void inputMatrix(int matrix[][5], int N): will take input in an NxN (where N <=5) matrix,
   - void rotate90Clockwise(int matrix[][5], int N): rotates the matrix by 90 degrees clockwise and
   - void displayMatrix(int matrix[][5], int N): displays the updated matrix.

   Example Input:
   Enter elements of the matrix:
   1 2 3
   4 5 6
   7 8 9
   After rotation and display functions, the expected output:
   7 4 1
   8 5 2
   9 6 3
5. A Sudoku array is a 9x9 matrix that follows these four rules:
   - Each row must contain all digits 1 through 9 exactly once.
   - Each column must contain all digits 1 through 9 exactly once.
   - The grid is divided into nine 3×3 subgrids, and each subgrid must also contain 1 through 9 exactly once.
   - No duplicates are allowed in any row, column, or 3×3 box.

   Your task is to implement a function **bool isSudokuArray(int grid[9][9])** that takes in a 9x9 array and determines if it is a Sudoku array.
   Example of Sudoku array. Function returns true.

4	3	5	2	6	9	7	8	1
6	8	2	5	7	1	4	9	3
1	9	7	8	3	4	5	6	2
8	2	6	1	9	5	3	4	7
3	7	4	6	8	2	9	1	5
9	5	1	7	4	3	6	2	8
5	1	9	3	2	6	8	7	4
2	4	8	9	5	7	1	3	6
7	6	3	4	1	8	2	5	9

{
{5,3,4,6,7,8,9,1,2},
{6,7,2,1,9,5,3,4,8},
{1,9,8,3,4,2,5,6,7},
{8,5,9,7,6,1,4,2,3},
{4,2,6,8,5,3,7,9,1},
{7,1,3,9,2,4,8,5,6},
{9,6,1,5,3,7,2,8,4},
{2,8,7,4,1,9,6,3,5},
{3,4,5,2,8,6,1,7,9}
}
	{
	{5,3,4,6,7,8,9,1,2},
	{6,7,2,1,9,5,3,4,8},
	{1,9,8,3,4,2,5,6,7},
	{8,5,9,7,6,1,4,2,3},
	{4,2,6,8,5,3,7,9,1},
	{7,1,3,9,2,4,8,5,6},
	{9,6,1,5,3,7,**2,8,4**},
	{2,8,7,4,1,9,**6,3,5**},
	{3,4,5,2,8,6,**1,7,5**}
	}

Return true	Return false. Duplicate "5" and missing “9” in last row. Missing “9” in the bottom rightmost 3x3 subgrid.